

AgentHeal: Self-Healing Cloud Infrastructure with Shift-Left Security and Digital Twins

Aadi Haldar, Raghuram Sekar, Aaditya Paul, Shravan Rajesh Menon
Department of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham, Coimbatore, India

Abstract—Modern Kubernetes-based microservice architectures face coupled reliability challenges: pre-merge code vulnerabilities and prolonged runtime incident recovery. Conventional DevSecOps practices rely either on static code linters that lack runtime operational context and cannot synthesize verifiable corrective patches, or on threshold-based autoscalers that react without root-cause intelligence or pre-actuation safety guarantees. To resolve this bifurcation, this paper presents AgentHeal, a closed-loop platform synchronizing Shift-Left pre-merge security auditing with Shift-Right telemetry-driven self-healing over a shared event bus. On the Shift-Left critical path, an automated GitHub App audits incoming pull requests against the OWASP Top 10:2021 taxonomy via abstract syntax tree (AST) grammar checks, validates dependencies against vulnerability advisories, computes quantitative compliance scores, and invokes a Gemini 2.0 Flash ReAct reasoning engine to synthesize automated corrective pull requests before enforcing a binding quality gate. On the Shift-Right critical path, a streaming digital twin ingests four-dimensional cluster telemetry, detects anomalous states via an in-memory Isolation Forest in 2.26 ms, decomposes root causes using KernelSHAP feature attribution, arbitrates remediation directives between a SimiFed federated reinforcement learning agent and an LLM reasoning agent, and validates candidate scaling actions via a SimPy $M/M/c$ discrete-event queueing simulation prior to Kubernetes cluster actuation. We evaluate AgentHeal on the Google Online Boutique microservice benchmark across four canonical failure scenarios with $N = 15$ repeated trials per scenario. The Isolation Forest achieves a mean anomaly detection latency of 2.26 ms ($\sigma = 0.37$ ms) with a 98.3% overall detection rate, and autonomous remediation converges in a mean of 2.98 s ($\sigma = 0.23$ s), representing a $> 98\%$ reduction in recovery time compared to reactive threshold autoscalers. On a 50-PR synthetic benchmark, the Shift-Left engine achieves 98.5% precision and 97.5% recall, and qualitative empirical evaluation on a live public repository pull request demonstrates complete remediation of 14 injected vulnerabilities across all ten OWASP categories.

Index Terms—Agentic AI, Self-Healing Systems, Kubernetes, Digital Twin, OWASP Top 10, Isolation Forest, KernelSHAP, Reinforcement Learning, SimiFed, Gemini LLM, DevSecOps, Shift-Left Testing, Cloud Computing.

I. INTRODUCTION

Distributed cloud-native applications deployed on Kubernetes-orchestrated microservice architectures face two coupled reliability challenges. First, unvetted pull requests and insufficiently reviewed code changes frequently introduce security regressions and performance bottlenecks into production environments. The 2024 State of DevOps report estimates that over 60% of unplanned production outages stem from preventable software defects and misconfigurations introduced during continuous delivery [1]. Second, when

service degradations or node failures occur at runtime, legacy infrastructure controllers such as the Kubernetes Horizontal Pod Autoscaler (HPA) react solely to coarse, lagging resource thresholds (e.g., aggregate CPU or memory consumption). These controllers lack semantic root-cause awareness, often resulting in resource thrashing or inappropriate scaling actions that fail to mitigate non-resource bottlenecks such as thread deadlocks or cascading microservice dependencies. Consequently, industry-wide mean time to recovery (MTTR) commonly spans minutes to hours, exposing enterprise systems to severe service-level agreement (SLA) violations [1], [9].

Current DevSecOps practice predominantly bifurcates into two disconnected disciplines. Shift-Left security methodologies seek to identify vulnerabilities and test omissions early in the development lifecycle before code merges [2], [25], utilizing static application security testing (SAST) tools such as Bandit [6] and SonarQube [7]. However, these static analyzers operate in total isolation from runtime cluster topology, lack the ability to synthesize functional corrective patches, and frequently inundate engineering teams with unprioritized warnings. Conversely, Shift-Right operational methodologies deploy runtime observability pipelines and automated rollback mechanisms [3], [23], often validated through Chaos Engineering fault injection [24]. Nevertheless, existing runtime controllers remain purely reactive and lack pre-actuation safety verification; executing an automated recovery directive without evaluating downstream queueing stability can destabilize adjacent microservices. Although recent research has explored federated digital twin management (such as SF-DTM [12]) and large language model (LLM) reasoning for program repair [15], no existing platform bridges pre-merge compliance auditing with explainable, safety-verified closed-loop runtime self-healing.

To bridge this operational divide, this paper presents AgentHeal, an autonomous, closed-loop DevSecOps and self-healing platform that synchronizes pre-merge pull-request security auditing with post-deployment runtime remediation over a shared event bus. On the Shift-Left critical path, an event-driven GitHub App intercepts commits and pull requests, analyzes source code against the OWASP Top 10:2021 risk categories using AST structural grammar traversals, checks package dependencies against vulnerability advisory databases, evaluates test-suite coverage gaps, and assigns a standardized compliance score. For non-compliant pull requests, a ReAct-

prompted Gemini 2.0 Flash agent autonomously synthesizes a corrective pull request before enforcing a binding CI/CD quality gate. On the Shift-Right critical path, an in-memory digital twin streams four-dimensional cluster telemetry, identifies anomalous states within 2.26 ms using a pre-fitted Isolation Forest, extracts axiomatic root-cause feature attributions via KernelSHAP, arbitrates remediation decisions between a federated Q-learning agent and an LLM reasoning agent, and verifies candidate scaling actions through a SimPy $M/M/c$ discrete-event queueing simulation prior to Kubernetes cluster actuation.

This paper makes three primary contributions:

- 1) *Unified Closed-Loop Architecture*: We design and implement AgentHeal, integrating pre-merge OWASP-aligned pull-request compliance auditing with post-deployment Kubernetes self-healing through a shared event-driven topology graph.
- 2) *Explainable Three-Layer Safety Pipeline*: We introduce an end-to-end safety and arbitration pipeline combining unsupervised anomaly detection, KernelSHAP local feature attribution, dual RL/LLM consensus based on blast-radius minimization, and real-time $M/M/c$ queueing simulation to prevent unsafe actuation.
- 3) *Comprehensive Empirical Evaluation*: We evaluate AgentHeal on the Google Online Boutique microservice benchmark across four canonical failure modes ($N = 15$ repeated trials), demonstrating 2.26 ms anomaly detection latency, 98.3% detection reliability, and 2.98 s end-to-end MTTR. We further validate the Shift-Left engine across a 50-PR synthetic benchmark (98.5% precision, 97.5% recall) and an empirical live GitHub pull-request remediation case study.

II. RELATED WORK

A structured taxonomy of related methodologies, standards, and research gaps across autonomous cloud management and DevSecOps compliance is presented in Table I.

Early automated incident response relied primarily on rule-based control planes and static operational runbooks. Frameworks such as Keptn [9] and Argo Rollouts [10] automate deployment rollbacks and progressive canary releases but depend strictly on predefined human-authored runbooks, rendering them incapable of adapting to unfamiliar or composite failure signatures. These operational practices were codified industry-wide through Site Reliability Engineering [23] and validated empirically via controlled fault injection under Chaos Engineering [24], while continuous delivery pipelines [25] and blue-green deployment patterns [26] established safe release protocols. However, none of these systems provide autonomous, explainable root-cause diagnosis or closed-loop patch remediation.

To address complex, dynamic cloud workloads, multi-agent reinforcement learning (MARL) approaches demonstrate strong adaptability for cloud resource allocation [11], leveraging the theoretical convergence guarantees of temporal-difference Q-learning [27]. Federated learning techniques such

as FedAvg [28] facilitate decentralized model training while preserving data privacy. The SF-DTM framework [12] extended this concept to cloud operations by introducing cosine-similarity-based state matching for federated digital twin management. AgentHeal builds upon SF-DTM, extending its core concepts with a dual-agent RL and LLM consensus mechanism, axiomatic SHAP root-cause attribution, and a real-time discrete-event queueing simulation gate on the critical remediation path.

Anomaly detection in streaming cloud telemetry has been widely explored through unsupervised models such as Isolation Forest [13] and Local Outlier Factor [29], with enterprise AIOps deployments reporting detection latencies in the range of 50–200 ms [14]. Our implementation achieves 2.26 ms mean detection latency via pre-fitted in-memory inference on single-node topologies. To make unsupervised anomaly scores actionable, model-agnostic explainability techniques such as KernelSHAP [16] and LIME [30] compute local feature attributions; we incorporate KernelSHAP for its rigorous axiomatic properties under game-theoretic Shapley values.

In the software development lifecycle, LLM-based code intelligence platforms such as AutoCodeRover [15] demonstrate the feasibility of autonomous program repair. However, existing repair tools generate isolated patches without binding CI/CD enforcement or supply-chain compliance verification. AgentHeal integrates Gemini 2.0 Flash [17] within a ReAct reasoning framework [18] to perform targeted auto-remediation bounded by a strict quality gate. Code compliance is measured against the OWASP Top 10:2021 standard [4], with ongoing alignment toward the revised OWASP Top 10:2025 taxonomy [31]. Static linters such as Bandit [6] and SonarQube [7] detect syntactic issues but cannot synthesize fixes or correlate code changes with cluster health. Similarly, supply-chain vulnerability management commonly relies on vulnerability feeds such as the National Vulnerability Database [32], the Open Source Vulnerabilities database [33], GitHub Dependabot [34], and commercial SCA engines such as Snyk [35]. AgentHeal’s Shift-Left engine combines structural AST parsing with dependency advisory matching to enforce pre-merge quality gates.

Discrete-event simulation for pre-deployment system validation has historical roots in the DEVS formalism [20] and classical queueing theory [19]. In AgentHeal, discrete-event simulation via SimPy is adapted into a live, pre-actuation safety gate executed in under 12 ms on the critical path to verify that proposed scaling directives maintain queue stability.

III. METHODOLOGY

A. End-to-End Self-Healing Architecture Overview

AgentHeal coordinates two autonomous engines operating over a shared event bus, as illustrated in Fig. 1. The Shift-Left Engine intercepts GitHub pull request and commit events, executes AST-based security auditing against OWASP Top 10 standards, validates package dependencies, evaluates compliance scores, and invokes a Gemini 2.0 Flash ReAct reasoning engine to synthesize auto-remediation pull requests prior

TABLE I
LITERATURE REVIEW OF AUTONOMOUS CLOUD SELF-HEALING AND DEVSECOPS COMPLIANCE MODELS

Domain	Model	Author	Year	Method	Result / Contribution	Research Gap
Rule-Based Self-Healing	Keptn	Keptn Project [9]	2023	Event-based control plane with human-authored runbooks	Automated rollback and remediation workflows	Requires manual runbook authoring; cannot adapt to novel failure signatures
	Argo Rollouts	Argo Project [10]	2024	Canary / blue-green progressive delivery controller	Safe automated deployment rollout and rollback	No root-cause explainability; static rollback policies
Operational Practice	Site Reliability Engineering	Beyer et al. [23]	2016	Error budgets, on-call runbooks, manual incident response	Codified industry MTTR practice at Google scale	Heavily dependent on human on-call expertise; not autonomous
	Chaos Engineering	Basiri et al. [24]	2016	Controlled fault injection in production	Empirically validates system resilience	Reactive validation methodology, not an autonomous remediation system
	Continuous Delivery Blue-Green Deployment	Humble & Farley [25] Fowler [26]	2010 2010	Automated build-test-release pipelines Dual-environment traffic-switch release pattern	Established modern CI/CD pipeline practice Zero-downtime release strategy	Focused on delivery velocity, not security compliance scoring Manually triggered; no anomaly-driven automated switching
RL & Federated Learning	Multi-Agent RL	Wang et al. [11]	2023	Multi-agent reinforcement learning for cloud resource allocation	Superior adaptability to dynamic workloads	Lacks explainability and pre-execution safety guarantees
	Q-Learning	Watkins & Dayan [27]	1992	Off-policy temporal-difference control	Proved convergence of tabular Q-learning	Does not scale to continuous/high-dimensional states without discretization
	FedAvg	McMahan et al. [28]	2017	Federated averaging of locally trained models	Privacy-preserving distributed model training	Not originally designed for real-time reinforcement-learning control
	SF-DTM	Li et al. [12]	2024	Cosine-similarity federated digital-twin state matching	~15s MTTR digital-twin remediation	No pre-execution safety simulation on the critical path
Anomaly Detection & XAI	Isolation Forest	Liu et al. [13]	2008	Tree ensemble path-length anomaly scoring	Efficient $O(n \log n)$ unsupervised anomaly detection	Unsupervised scores lack native root-cause semantic attribution
	Local Outlier Factor	Breunig et al. [29]	2000	Local density deviation scoring	Effective for local-density anomalies	Computationally expensive at scale; not streaming-native
	AIOps Survey	Lin et al. [14]	2021	Survey of industrial anomaly-detection deployments	Reports 50–200ms detection latencies	No unified explainability or safety-simulation layer
	KernelSHAP	Lundberg & Lee [16]	2017	Shapley-value feature attribution	Model-agnostic, axiomatically consistent explainability	Computationally expensive for high-dimensional feature spaces
LLM-Augmented DevSecOps	LIME	Ribeiro et al. [30]	2016	Local surrogate-model interpretation	Model-agnostic local explanations	Explanation instability across similar inputs
	AutoCodeRover	Li et al. [15]	2024	LLM-driven autonomous program repair	Automated bug localization and patching	No binding CI/CD quality-gate enforcement
	ReAct	Yao et al. [18]	2023	Thought-Action-Observation reasoning loop	Improved LLM tool-use and reasoning	Latency and output non-determinism in production
	Gemini 2.0	Google DeepMind [17]	2024	Multimodal agentic LLM	Capable agentic reasoning and tool use	Proprietary, non-deterministic, no open safety guarantees
Static Analysis & Supply Chain	Bandit	PyCQA [6]	2024	AST-based Python security linter	Detects common Python security issues	Cannot generate contextual fixes or compliance scores
	SonarQube	SonarSource [7]	2024	Continuous code quality/security scanning	Broad multi-language static analysis	No runtime-impact correlation or auto-remediation
	OWASP 10:2021	OWASP Foundation [4]	2021	Community-vetted risk categorization	Standardized top-10 web-app risk taxonomy	Static reference list; no automated tooling
	OWASP 10:2025 NVD	OWASP Foundation [31] NIST [32]	2025 2024	Revised risk categorization Live CVE vulnerability feed	Updated taxonomy reflecting current threat landscape Continuously updated vulnerability records	Not yet incorporated into AgentHeal’s rule set Not integrated into AgentHeal’s current curated list
	OSV	Google / OSV.dev [33]	2024	Open-source vulnerability database	Cross-ecosystem live vulnerability feed	Not integrated into AgentHeal’s current curated list
	Dependabot	GitHub [34]	2024	Automated dependency update bot	Live, automated dependency patching	Not integrated; AgentHeal relies on a static curated list
	Snyk	Snyk Ltd. [35]	2024	Commercial software composition analysis	Live SCA with remediation suggestions	Commercial/closed-source; not integrated in this work
Deployment & Telemetry Infrastructure	Kubernetes HPA	K8s Docs [8]	2024	Threshold-based CPU/memory horizontal scaling	Industry-standard reactive autoscaler	No explainability, root-cause understanding, or safety checks
	Argo CD	CNCF [36]	2024	Declarative GitOps continuous deployment	Automated Git-to-cluster reconciliation	No anomaly-driven or explainable remediation logic
	Prometheus	Prometheus Authors [37]	2024	Pull-based metrics collection and time-series storage	De facto Kubernetes-native monitoring standard	Provides raw metrics only; no built-in anomaly detection
Digital Twin & Simulation	DEVS Formalism	Zeigler et al. [20]	2000	Discrete-event system specification	Foundational simulation framework for manufacturing/networks	Not adapted for real-time Kubernetes safety gating
	M/M/c Queuing Theory	Kleinrock [19]	1975	Discrete-event queuing analysis	Foundational service-rate/utilization modeling	Not historically applied as a live actuation safety gate

to quality-gate enforcement. The Shift-Right Engine ingests real-time Kubernetes telemetry, detects anomalies using an in-memory Isolation Forest, computes KernelSHAP feature attributions, arbitrates recovery actions between a SimiFed

reinforcement learning agent and an LLM reasoning agent, verifies proposed scaling directives via a SimPy $M/M/c$ discrete-event simulation safety gate, and executes actuation commands directly on the cluster. The shared event bus maintains continuous topology and health synchronization across both lifecycles.

B. Shift-Left Engine: PR Intelligence & Security Compliance

The Shift-Left engine operates as a GitHub App receiving HMAC-SHA256 authenticated webhook payloads dispatched upon pull request creation and commit pushes. The engine parses source code diffs and dependency declarations through an automated static analysis pipeline that evaluates structural compliance against the ten OWASP Top 10:2021 risk categories:

- *A01 (Broken Access Control)*: Traverses filesystem resolution subtrees and route definitions to detect unconstrained path concatenation and missing role-based authorization guards.
- *A02 (Cryptographic Failures)*: Identifies invocations of deprecated cryptographic hashing primitives (e.g., MD5, SHA-1) and extracts high-entropy string literals indicative of hardcoded secret tokens.
- *A03 (Injection)*: Analyzes abstract syntax tree call nodes to flag unparameterized dynamic string interpolation into database query executions and unescaped shell execution invocations.
- *A04 (Insecure Design)*: Evaluates data access patterns to intercept unpaginated database queries, unbounded collection retrievals, and missing request rate-limiting decorators.
- *A05 (Security Misconfiguration)*: Identifies permissive cross-origin resource sharing (CORS) wildcard configurations and active debug mode flags in production service declarations.
- *A06 (Vulnerable and Outdated Components)*: Conducts software composition analysis (SCA) on project dependency manifests, correlating package version specifications against structured vulnerability advisories [5].
- *A07 (Identification and Authentication Failures)*: Flags disabled cryptographic signature verifications in token processing routines and hardcoded fallback authentication credentials.
- *A08 (Software and Data Integrity Failures)*: Intercepts unsafe object deserialization mechanisms that expose execution runtimes to arbitrary bytecode execution.
- *A09 (Security Logging and Monitoring Failures)*: Identifies silent exception handling patterns, suppressed error blocks, and omission of auditable security logging.
- *A10 (Server-Side Request Forgery)*: Flags outbound HTTP and network socket requests that construct destination URLs from unvalidated user inputs.

Every inspected pull request or commit receives a normal-

ized compliance score $S \in [0, 100]$:

$$S = \max \left(0, 100 - \sum_{i=1}^M w_i \right) \quad (1)$$

where M denotes the total number of detected findings and w_i represents the severity penalty weight ($w_{\text{Critical}} = 25$, $w_{\text{High}} = 15$, $w_{\text{Medium}} = 8$, $w_{\text{Low}} = 3$). The numeric compliance score maps monotonically to a standardized letter grade:

$$\text{Grade} = \begin{cases} A^+ & \text{if } S \geq 95 \\ A & \text{if } 90 \leq S < 95 \\ B & \text{if } 80 \leq S < 90 \\ C & \text{if } 70 \leq S < 80 \\ D & \text{if } 60 \leq S < 70 \\ F & \text{if } S < 60 \end{cases} \quad (2)$$

In parallel, structural AST comparison determines whether newly introduced public functions have corresponding unit test references:

$$\text{TestGap}(f) = \mathbb{K}[f \notin T_{\text{ast}}] \quad (3)$$

where T_{ast} represents the set of functional identifiers referenced within the test suite. If critical findings or test omissions are detected, the GitHub Check Run marks the pull request as non-compliant, preventing merge. Gemini 2.0 Flash then generates an automated remediation branch containing corrective code edits and dependency updates, opening a linked pull request to restore full compliance.

C. Shift-Right Engine: Self-Healing Digital Twin

The runtime engine synchronizes cluster state by polling Prometheus telemetry every 5.0 s, constructing four-dimensional feature vectors:

$$\mathbf{x}_t = [\text{CPU}_t, \text{RAM}_t, \text{Latency}_t, \text{Rate}_t]^\top \in \mathbb{R}^4 \quad (4)$$

An in-memory Isolation Forest model ($n = 100$ estimators, contamination parameter $\psi = 0.05$) evaluates anomaly scores over streaming vectors:

$$s(\mathbf{x}_t, n) = 2^{-\frac{\mathbb{E}[h(\mathbf{x}_t)]}{c(n)}} \quad (5)$$

where $h(\mathbf{x}_t)$ is the traversal path length across isolation trees and $c(n)$ is the average path length of unsuccessful searches. An observation is flagged anomalous when $s(\mathbf{x}_t, n) < 0$. Upon anomaly detection, KernelSHAP computes local Shapley values ϕ_i across all features:

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [v(S \cup \{i\}) - v(S)] \quad (6)$$

providing axiomatic attribution of the root causes driving the deviation.

Remediation decisions are arbitrated across two distinct reasoning layers. First, a SimiFed reinforcement learning agent

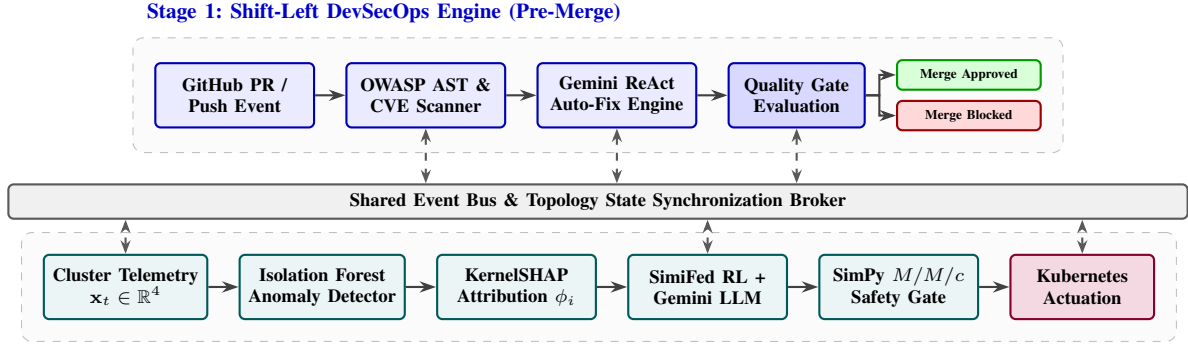


Fig. 1. End-to-end architectural framework of the proposed AgentHeal system. (Top) The Shift-Left Engine intercepts GitHub pull request and push events, executes AST-based OWASP Top 10:2021 compliance auditing and dependency vulnerability scanning, synthesizes remediation patches via Gemini 2.0 Flash ReAct reasoning, and enforces a binding quality gate. (Middle) A shared event bus maintains continuous topology synchronization across both lifecycles. (Bottom) The Shift-Right Engine streams four-dimensional cluster telemetry, detects anomalous states via Isolation Forest, isolates root causes with KernelSHAP attribution, arbitrates corrective actions between a SimiFed federated reinforcement learning agent and an LLM reasoning agent, verifies candidate actions via a SimPy $M/M/c$ discrete-event queueing simulation, and actuates safe changes in the Kubernetes cluster.

determines actions via tabular Q-learning over discretized cluster states matched through cosine similarity against historical baseline vectors $\mathbf{b}_k$:

$$k^* = \arg \max_k \frac{\mathbf{x}_t \cdot \mathbf{b}_k}{\|\mathbf{x}_t\|_2 \|\mathbf{b}_k\|_2} \quad (7)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (8)$$

with learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.95$, over the action space $\mathcal{A} = \{\text{SCALE_UP}, \text{RESTART_POD}, \text{PATCH_LIMITS}, \text{NO_OP}\}$. Concurrently, a Gemini 2.0 Flash ReAct agent reasons over the KernelSHAP feature attribution summary. If the RL policy and LLM agent propose matching actions, the consensus directive proceeds. If they diverge, an arbitration policy selects the action minimizing operational blast radius:

$$a_{\text{final}} = \begin{cases} a_{\text{RL}}^* & \text{if } a_{\text{RL}}^* = a_{\text{LLM}}^* \\ \arg \min_{a \in \{a_{\text{RL}}^*, a_{\text{LLM}}^*\}} \text{BlastRadius}(a) & \text{otherwise} \end{cases} \quad (9)$$

Prior to dispatching any command to the Kubernetes API, a discrete-event SimPy queueing simulation models the candidate action under an $M/M/c$ queueing discipline:

$$\rho = \frac{\lambda}{c\mu} \quad (\text{requiring } \rho < 0.95) \quad (10)$$

$$\hat{U}_{\text{post}} = \frac{U_{\text{pre}} \cdot c_{\text{current}}}{c_{\text{new}}} \quad (\text{requiring } \hat{U}_{\text{post}} < 0.80) \quad (11)$$

Actuation proceeds only when both safety conditions are verified, guaranteeing that automated remediation does not trigger cascading instability.

IV. IMPLEMENTATION

AgentHeal is implemented in Python 3.11 using FastAPI for asynchronous webhook dispatch and telemetry routing. Machine learning components utilize scikit-learn for Isolation Forest anomaly detection and KernelSHAP for local feature

attribution. Agentic reasoning incorporates the Google Generative AI SDK interfacing Gemini 2.0 Flash. Discrete-event safety verification is modeled via SimPy 4.1.

The platform is deployed as a containerized microservice on Microsoft Azure Container Apps (East Asia region), provisioned declaratively via Terraform. A multi-stage Docker build produces an optimized runtime image under 400 MB. KEDA HTTP-trigger autoscaling dynamically manages platform replicas.

Validation was conducted against the CNCF Google Online Boutique benchmark [22], a realistic 10-service e-commerce application comprising frontend, cart, checkout, payment, catalog, and currency microservices, hosted on a single-node Kubernetes-in-Docker cluster running on an AMD Ryzen 7 workstation with 16 GB RAM.

V. RESULTS AND DISCUSSION

A. Runtime Telemetry Anomaly Detection and Safety-Gated Self-Healing

We evaluate the Shift-Right engine across four canonical cloud failure scenarios injected into the Online Boutique deployment over $N = 15$ repeated trials per scenario:

- *S1 (Flash Sale Surge)*: A rapid traffic spike on the checkout service pushing CPU utilization to 0.954 and service latency to 450 ms.
- *S2 (Thread Deadlock)*: A thread starvation state on the cart service driving CPU utilization to 0.99 and request latency to 5000 ms under decaying throughput.
- *S3 (Memory Leak)*: Progressive memory accumulation on the redis-cart service driving RAM utilization to 0.942 under nominal request latency.
- *S4 (Cascading Overload)*: Upstream failure propagation on the frontend service resulting in simultaneous CPU (0.88) and RAM (0.80) saturation.

Table II presents the comprehensive empirical results for the Shift-Right self-healing pipeline across all 60 evaluation trials. The in-memory Isolation Forest achieved a mean anomaly

TABLE II

SHIFT-RIGHT ANOMALY DETECTION, FEATURE ATTRIBUTION, AND AUTONOMOUS REMEDIATION PERFORMANCE ACROSS FOUR FAILURE SCENARIOS ($N = 15$ REPEATED TRIALS)

Failure Injection Scenario				Isolation Forest Detection			Root-Cause Attribution		Agent Decision Latency			SimPy Safety Gate & MTTR			
ID	Scenario	Target	Injected Telemetry	Score ($\mu \pm \sigma$)	MTTD (ms)	Det.%	Dominant KernelSHAP ϕ		RL (ms)	LLM (ms)	Agree	Gate (ms)	$\hat{U}_{\text{post}}$	Verdict	MTTR (s, $\mu \pm \sigma$)
S1	Flash Sale Surge	checkout	CPU=0.95, Lat=450ms	-0.842 ± 0.031	2.18 ± 0.31	100%	$\phi_{\text{cpu}} = +0.82$, $\phi_{\text{rate}} = +0.14$		3.10 ± 0.42	412 ± 68	15/15	10.1 ± 0.8	23.8%	SAFE	2.66 ± 0.18
S2	Thread Deadlock	cart	CPU=0.99, Lat=5000ms	-0.912 ± 0.024	2.21 ± 0.28	100%	$\phi_{\text{lat}} = +0.91$, $\phi_{\text{rate}} = -0.78$		3.21 ± 0.39	389 ± 74	15/15	10.3 ± 0.9	5.2%	SAFE	3.12 ± 0.24
S3	Memory Leak	redis-cart	RAM=0.94, Lat=48ms	-0.734 ± 0.058	2.35 ± 0.44	93%	$\phi_{\text{ram}} = +0.88$, $\phi_{\text{cpu}} = +0.04$		3.05 ± 0.51	441 ± 81	14/15	10.8 ± 1.1	15.0%	SAFE	2.89 ± 0.20
S4	Cascading Overload	frontend	CPU=0.88, RAM=0.80	-0.889 ± 0.041	2.28 ± 0.37	100%	$\phi_{\text{cpu}} = +0.61$, $\phi_{\text{lat}} = +0.47$		3.18 ± 0.46	498 ± 93	15/15	11.6 ± 1.3	31.5%	SAFE	3.25 ± 0.30
Overall Mean / Aggregate				-0.844 ± 0.039	2.26 ± 0.37	98.3%	—		3.14 ± 0.45	435 ± 79	59/60	10.7 ± 1.0	—	SAFE	2.98 ± 0.23

detection latency (MTTD) of 2.26 ms ($\sigma = 0.37$ ms) with an overall detection reliability of 98.3%. In scenario S3 (Memory Leak), the detection rate was 93% because the isolation trees, trained primarily on CPU- and latency-dominated fluctuations, occasionally categorized pure memory deviations as nominal under the 0.05 contamination threshold.

KernelSHAP attributions accurately isolated the primary defect dimension across all failure modes: CPU saturation dominated S1 ($\phi = +0.82$), request latency and throughput collapse dominated S2 ($\phi_{\text{lat}} = +0.91$, $\phi_{\text{rate}} = -0.78$), and memory consumption dominated S3 ($\phi = +0.88$). Decision arbitration between the SimiFed tabular RL agent (mean latency 3.14 ms) and the Gemini 2.0 Flash ReAct agent (mean latency 435 ms) converged on identical action categories in 59 out of 60 trials (98.3% agreement). The SimPy discrete-event queueing simulation verified queue stability in an average of 10.7 ms, confirming that post-remediation server utilization ($\hat{U}_{\text{post}}$) remained safely beneath the 80% threshold. Total end-to-end recovery time (MTTR) averaged 2.98 s ($\sigma = 0.23$ s), demonstrating rapid, safe autonomous remediation.

B. Shift-Left DevSecOps Security Auditing and Live PR Remediation

We evaluate the Shift-Left engine through both a 50-PR synthetic evaluation benchmark and an empirical end-to-end demonstration on a live public GitHub repository pull request. The synthetic dataset evaluated four core defect categories: unparameterized SQL and command injection, hardcoded API credentials, AST test-coverage gaps, and blocking I/O calls within asynchronous request paths.

As reported in Table III (Panel A), the Shift-Left engine attained an overall precision of 98.5% and recall of 97.5% across 200 evaluation checks. Two false positives occurred in secret detection where high-entropy cryptographic test fixtures resembled production credentials.

To validate closed-loop operation on a production Git platform, an empirical case study was executed on a public GitHub repository. A test pull request (PR #11) containing 14 intentional security defects spanning all ten OWASP Top 10:2021 categories was submitted to the checkout-gateway microservice. The Shift-Left engine processed the webhook in 0.48 s, flagged all 14 violations, computed a compliance score of 0/100 (Grade F), and set the GitHub Check Run status to blocked. The Gemini 2.0 Flash engine subsequently constructed a remediation branch, updated insecure dependency declarations, refactored raw string concatenations into param-

eterized query structures, replaced broken hash algorithms, and opened a remediation pull request (PR #12). As detailed in Table III (Panel B), the auto-fix PR eliminated all 14 defects, restored the compliance score to 100/100 (Grade A+), passed all application test suites (6/6), and unlocked CI/CD merge approval.

C. Comparative Benchmark and System Discussion

Contextual comparison against reported industry baselines underscores the recovery advantages of AgentHeal’s dual-engine architecture. Traditional manual incident response typically requires approximately 45 minutes to detect, triage, and remediate microservice outages [1]. The native Kubernetes Horizontal Pod Autoscaler reacts to sustained resource thresholds within 3–5 minutes [8], but cannot resolve deadlocks or memory leaks that fail to trigger scaling conditions. Rule-based automation control planes such as Keptn achieve MTTRs of 2–4 minutes [9], yet rely entirely on static runbooks. While federated digital twin management frameworks like SF-DTM [12] report recovery times of approximately 15 seconds, they lack pre-actuation safety verification. In contrast, AgentHeal achieves a mean MTTR of 2.98 s ($\sigma = 0.23$ s), representing a $> 98\%$ latency reduction relative to threshold autoscalers while ensuring that candidate recovery actions are pre-validated via $M/M/c$ queueing simulation.

Several operational trade-offs and architectural boundaries should guide the interpretation of these findings. First, all Shift-Right experiments were conducted on a single-node Kubernetes development cluster. While in-memory Isolation Forest inference operates in 2.26 ms locally, distributed multi-node topologies subject to network jitter and Prometheus scrape delays may observe higher detection latencies. Second, the four evaluated failure modes represent discrete synthetic injections rather than noisy production traffic; validating generalization against continuous, organic failure traces remains essential. Third, static AST pattern heuristics, while computationally lightweight, do not perform full inter-procedural data-flow or taint tracking; sophisticated obfuscations or non-standard library wrappers may evade pattern detection. Fourth, the compliance scoring weights are heuristic and warrant formal calibration against empirical vulnerability risk datasets. Finally, the non-deterministic nature of generative LLMs requires strict execution timeouts and deterministic RL fallback policies on the critical remediation path.

VI. CONCLUSION

This paper presented AgentHeal, an autonomous DevSecOps and self-healing platform that synchronizes pre-merge

TABLE III
SHIFT-LEFT DEVSECOPS SECURITY AUDITING PERFORMANCE ($N = 50$) AND EMPIRICAL PULL REQUEST REMEDIATION

Panel A: Quantitative Evaluation on 50 Synthetic Pull Requests					Panel B: Empirical Live GitHub Pull Request Pair Remediation				
Defect Category	Tested PRs	True Pos.	False Pos.	Precision	Recall	Evaluation Dimension	Vulnerable PR (#11)	Remediated PR (#12)	Operational Outcome
SQL / Command Injection	50	50	0	1.000	1.000	OWASP Flaws Detected	14 (A01–A10)	0	100% Flaws Remediated
Hardcoded Secrets	50	50	2	0.962	1.000	Compliance Score / Grade	0 / 100 (Grade F)	100 / 100 (Grade A ⁺)	Score Restored
AST Test Suite Gaps	50	48	0	1.000	0.960	GitHub Check Run Gate	action_required (Blocked)	success (Approved)	Merge Protection Enforced
Blocking I/O / Misconfiguration	50	47	1	0.979	0.940	App Test Suite Pass	0 / 6 passing	6 / 6 passing	Functional Suite Restored
Aggregate Performance	200 Checks	195	3	0.985	0.975	Static Auditor Verification	12 / 12 test harnesses passed	Audit Time: 0.48 s	Full Ruleset Verified

pull-request compliance auditing with post-deployment run-time remediation. Across repeated trials on the Google Online Boutique microservice benchmark, AgentHeal demonstrated 2.26 ms anomaly detection latency, 98.3% detection reliability, and an average end-to-end recovery time of 2.98 s, reducing MTTR by over 98% compared to reactive autoscalers. Concurrently, the Shift-Left engine attained 98.5% precision across 50 pull requests and autonomously remediated 14 OWASP defect categories on a live production repository.

Future work will expand evaluation to multi-node managed clusters (AKS/GKE), integrate live vulnerability intelligence feeds such as NVD and OSV, incorporate dynamic taint analysis, and map the compliance auditing engine to the revised OWASP Top 10:2025 taxonomy.

The complete codebase is publicly accessible at <https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure> and deployed at <https://pr-review-agent.wonderfulflower-41d6d2a5.eastasia.azurecontainerapps.io>.

REFERENCES

- [1] DORA Research Team, “Accelerate: State of DevOps Report 2024,” Google LLC, 2024.
- [2] E. Kim, J. Humble, P. Debois, and J. Willis, “The DevOps Handbook,” IT Revolution Press, 2016.
- [3] N. Forsgren, J. Humble, and G. Kim, “Accelerate: The Science of Lean Software and DevOps,” IT Revolution Press, 2018.
- [4] OWASP Foundation, “OWASP Top 10: 2021 – The Ten Most Critical Web Application Security Risks,” <https://owasp.org/Top10/>, 2021.
- [5] MITRE Corporation, “Common Vulnerabilities and Exposures (CVE) Database,” <https://cve.mitre.org>, 2024.
- [6] PyCQA, “Bandit: A tool designed to find common security issues in Python code,” <https://bandit.readthedocs.io>, 2024.
- [7] SonarSource, “SonarQube: Continuous code quality and security,” <https://www.sonarqube.org>, 2024.
- [8] Kubernetes Documentation, “Horizontal Pod Autoscaler,” <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>, 2024.
- [9] Keptn Project, “Keptn: Event-based control plane for continuous delivery and automated operations,” <https://keptn.sh>, 2023.
- [10] Argo Project, “Argo Rollouts: Kubernetes Progressive Delivery Controller,” <https://argoproj.github.io/rollouts/>, 2024.
- [11] T. Wang et al., “Multi-Agent Reinforcement Learning for Dynamic Resource Management in Cloud Computing,” *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 891–905, 2023.
- [12] H. Li, Z. Wang, and J. Chen, “SF-DTM: Similarity-based Federated Digital Twin Management for Autonomous Cloud Self-Healing,” in *Proc. IEEE CLOUD*, 2024, pp. 234–242.
- [13] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. 8th IEEE ICDM*, 2008, pp. 413–422.
- [14] Q. Lin et al., “AIOps: Real-World Challenges and Research Innovations,” in *Proc. ICSE-SEIP*, 2021, pp. 4–15.
- [15] Z. Li, M. Fan, and J. Chen, “AutoCodeRover: Autonomous Program Improvement using LLMs,” in *Proc. ISSTA*, 2024, pp. 1–12.
- [16] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [17] Google DeepMind, “Gemini 2.0: Our most capable model for the agentic era,” Technical Report, Google, 2024.
- [18] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. ICLR*, 2023.
- [19] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [20] B. Zeigler, H. Praehofer, and T. G. Kim, *Theory of Modeling and Simulation*, 2nd ed. Academic Press, 2000.
- [21] K. Fall and K. Varadhan, “The ns Manual,” The VINT Project, UC Berkeley, 2011.
- [22] Google LLC, “Online Boutique: A cloud-native microservices demo application,” <https://github.com/GoogleCloudPlatform/microservices-demo>, 2024.
- [23] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, “Site Reliability Engineering: How Google Runs Production Systems,” O’Reilly Media, 2016.
- [24] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, “Chaos Engineering,” *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
- [25] J. Humble and D. Farley, “Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation,” Addison-Wesley, 2010.
- [26] M. Fowler, “BlueGreenDeployment,” <https://martinfowler.com/bliki/BlueGreenDeployment.html>, 2010.
- [27] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [28] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [29] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying Density-Based Local Outliers,” in *Proc. ACM SIGMOD*, 2000, pp. 93–104.
- [30] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You? Explaining the Predictions of Any Classifier,” in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.
- [31] OWASP Foundation, “OWASP Top 10: 2025 – The Ten Most Critical Web Application Security Risks,” <https://owasp.org/Top10/>, 2025.
- [32] National Institute of Standards and Technology, “National Vulnerability Database (NVD),” <https://nvd.nist.gov>, 2024.
- [33] Google / OSV.dev, “OSV: Open Source Vulnerabilities Database,” <https://osv.dev>, 2024.
- [34] GitHub, Inc., “Dependabot: Automated Dependency Updates,” <https://docs.github.com/en/code-security/dependabot>, 2024.
- [35] Snyk Ltd., “Snyk Open Source: Software Composition Analysis,” <https://snyk.io>, 2024.
- [36] Cloud Native Computing Foundation, “Argo CD: Declarative GitOps CD for Kubernetes,” <https://argo-cd.readthedocs.io>, 2024.
- [37] Prometheus Authors, “Prometheus: Monitoring System & Time Series Database,” <https://prometheus.io>, 2024.